

What Does a Derivative Action Space Buy a Diffusion Policy?

A Controlled Map of First-Order Action Filters

Yufeng Li

[affiliation]

Abstract—Reparameterizing a robot policy’s action space—velocities instead of positions, accelerations instead of velocities, or a learned “derivative” command that a low-level controller integrates—is a recurring proposal with recurring claims: smoother execution, easier reinforcement-learning (RL) fine-tuning, better robustness, safer adaptation. We show that every such interface is a first-order filter $c_t = \lambda c_{t-1} + s u_t$ between the policy output u_t and the plant, differing from a plain low-pass filter only in its DC gain and in whether the filter state is exposed to the policy. We map this (λ, s) space on a diffusion-policy RL fine-tuning pipeline (robomimic Square, DPPO) along five axes: prior alignment, RL trainability, execution smoothness, deployment robustness, and self-supervised signal quality, using twelve trained configurations, three evaluation seeds per cell, and five training seeds for the two main arms. The map is built with a protocol we found necessary to avoid false positives: a matched-exploration-budget twin, the strongest raw baseline we could train, a low-pass null hypothesis, direct mechanistic measurements, and upstream-versus-downstream perturbation injection; each control is motivated by a wrong conclusion we would otherwise have drawn. On this map, the derivative interface buys a better behaviour-cloning prior—and the prior improves as the derivative scale s *shrinks*, the opposite of the common intuition—and it colours exploration noise. It does not improve fine-tuning, zero-shot drift robustness, or adaptation: trainability and robustness are functions of the executed exploration noise $\sigma_{\text{exec}} = s\sigma/\sqrt{1-\lambda^2}$ (Spearman $\rho = 0.76$ across ten configurations, 0.92 across the nine that exclude the original interface) rather than of filter structure, and at matched executed noise no interface beats the raw policy. The reason is an identity: when the policy observes the filter state—which the derivative parameterization requires for identifiable labels—closed-loop relabelling makes the interface an exact reparameterization of the position policy ($c_t = a_t$), so the filter can shape only what is added to the policy output, never the intended trajectory. Its corollaries hold on the map: the policy output is the demonstration pre-whitened by λ , deterministic execution has the demonstrations’ smoothness rather than the filter’s ($2.8\text{--}3.2\times$ the jerk of an unobservable-state twin at equal filter and noise), adaptive corrections applied upstream of the filter are absorbed, and tracking-residual signals lose identifiability. We release the map, the protocol, and the drift and misadaptation suites.

I. INTRODUCTION

A robot policy rarely commands the plant directly. Between the network output and the actuators sits an *action interface*: a position-delta command tracked by an operational-space controller, a velocity command integrated by the controller, an acceleration or “derivative” command that is integrated once more, or a learned command passed through a low-pass filter.

Changing this interface is cheap and is repeatedly proposed as a way to obtain smoother execution [1], better-shaped exploration noise [2], easier or more stable RL fine-tuning of imitation-learned policies [3], [4], robustness to actuator and sensor drift, and a safer place to perform deployment-time adaptation [5], [6]. These claims are usually supported by a single configuration compared against a single baseline.

We started from exactly such a proposal: a *derivative action interface* in which a diffusion policy [7] outputs the increment of the operational-space command, a leaky integrator accumulates it, and the integrator state is appended to the observation. The intended benefits were the standard list above. Over ten days and roughly nine hundred evaluation rollouts, each of the intended benefits either failed to materialize against a sufficiently strong control or turned out to be a property of a scalar noise quantity rather than of the interface. This paper reports what survived.

Our first observation is structural. Writing the interface as

$$c_t = \lambda c_{t-1} + s u_t, \quad \lambda \in [0, 1], s > 0, \quad (1)$$

where u_t is the policy output and c_t the executed command, the raw policy is the point $(\lambda, s) = (0, 1)$, a first-order low-pass filter with coefficient α is $(\alpha, 1 - \alpha)$ with unit DC gain, and a leaky “derivative” interface is any point with DC gain $s/(1 - \lambda) > 1$. The derivative interface therefore differs from a low-pass filter on a position-domain policy in exactly two respects: its DC gain, and the fact that its state c_{t-1} is observed by the policy (which, as we show, is necessary for the derivative labels to be identifiable from demonstrations). The question “what does a derivative action space buy” becomes a question about the (λ, s) plane, the training exploration noise σ , and one binary design choice.

Our second observation is methodological. The first version of our study compared the interface against a *matched-budget twin*—the raw policy fine-tuned with the same policy-side exploration noise σ —and found large advantages: +25 to +37 percentage points (pp) under actuator-gain drift, and a residual-driven gain compensation that “worked on the interface but not on the raw policy.” All of these advantages disappeared against the raw policy trained with the default exploration noise $\sigma = 0.10$, which is more robust than any interface configuration we trained (81% versus at most 46% under a 30% actuator-gain loss) and uses the same compensation perfectly. The twin was a weak policy, not a fair control. Along

the way we also (i) proposed and then refuted a mechanism (“integral action compensates multiplicative gain error”) by a direct measurement of the compensation ratio, (ii) discovered that our first adaptation experiments applied the correction *downstream* of the integrator and therefore never tested the structural bound they were meant to test, and (iii) shipped an estimator whose first batch of results was an artefact of a per-episode reset. We turned each of these into a rule of the evaluation protocol (Sec. IV).

Contributions.

- 1) **The map.** Twelve configurations on the $(\lambda, s) \times \sigma \times \{\text{observable state}\}$ space (Table I), each evaluated on five axes with three evaluation seeds; the two principal arms with five training seeds. Most action-space papers report one point and no controls.
- 2) **The protocol.** Five controls—matched-budget twin, strongest raw baseline, low-pass null hypothesis, direct mechanistic measurement, upstream-versus-downstream injection—each paired with the wrong conclusion it prevented in this study.
- 3) **Findings.** (a) *An identity.* With the filter state observed, closed-loop relabelling is an exact change of coordinates, $c_t = a_t$: the derivative interface is a position policy conditioned on its previous command with a rescaled target, and the filter acts on perturbations of the output, never on intent (Proposition 1). Its corollaries are verified on the map: the policy output is the demonstration pre-whitened by λ (measured in the relabelled datasets before any training), deterministic execution has the demonstrations’ jerk (0.07–0.13 versus 0.107) rather than the filter’s (0.03 for an unobservable low-pass), and corrections applied upstream of the filter are absorbed by the policy. (b) Trainability and zero-shot drift robustness are governed by the *executed* exploration noise during fine-tuning, $\sigma_{\text{exec}} = s\sigma/\sqrt{1-\lambda^2}$, and its chunk-scale, colour-aware variant, not by filter structure. High-DC-gain interfaces amplify noise and therefore force training at low σ ; this explains why the original interface could not be trained at the raw policy’s σ , but not all of its fragility—it and one other observable-state configuration sit below the noise trend, a residual we characterize but do not resolve. (c) The leaky integrator is a high-DC-gain low-pass filter; the scale s trades a tighter execution-space trust region against *nothing*: the behaviour-cloning prior *improves* as s shrinks (49%→58.5% from $s = 1$ to $s = 0.3$ at $\lambda = 0.9$), because s sets the signal-to-noise ratio of the derivative labels against the diffusion model’s output-noise floor. (d) Observing the filter state also blocks upstream adaptation and degrades residual-signal identifiability; the interface arm’s fine-tuning outcome has $6.5\times$ the training-seed variance of the raw arm at equal mean.

II. RELATED WORK

Action spaces and smoothing. Velocity, acceleration and delta parameterizations are standard in manipulation and driving; smoothing is obtained either at the policy (state-dependent exploration [1], coloured exploration noise [2], action chunking [4], [8]) or at the plant (low-pass filters, impedance control). Pink noise [2] establishes that a spectral exponent $\beta \approx 1$ is a good default for exploration in continuous control; we show that a leaky integrator implements a continuously tunable $\beta(\lambda)$ on the executed command but also that the same shaping caps the usable σ .

RL fine-tuning of diffusion policies. DPPO [3] fine-tunes the last denoising steps of a diffusion policy with PPO [9]; its exploration noise is the sampling standard deviation of those steps. Stability of offline-to-online transitions has been studied through value calibration [10], replay [11] and warm-starts [12]; action chunking [4] is the closest to an interface-level intervention. We take DPPO as a fixed pipeline and vary only the interface and σ .

Adaptation at deployment. Parameter-level online adaptation with stability guarantees [13], feed-forward adaptation [14], self-supervised policy adaptation [15], test-time training of robot policies [16]–[18] and weight-free steering [5], [6] all change something between observation and actuator. Our contribution to this literature is negative and structural: we measure where in the interface an adaptive correction can be applied at all, and show that the one place that would enjoy a propagation bound is unavailable when the filter state is observed.

Evaluation methodology. Our protocol is in the spirit of studies that question progress by strengthening baselines [19], [20]; we add controls specific to action-interface claims.

III. FIRST-ORDER ACTION FILTERS

A. Definitions

Let the operational-space controller accept a normalized command $c_t \in [-1, 1]^d$ per control step (position and rotation deltas; the gripper dimension bypasses the filter). A policy π outputs u_t ; the interface is (1) with clipping to $[-1, 1]$. The *raw* interface is $(\lambda, s) = (0, 1)$. A *low-pass* interface with coefficient α is $(\alpha, 1 - \alpha)$: unit DC gain, the policy trained in the position domain, the filter inserted at fine-tuning time. A *derivative* interface is (λ, s) with DC gain $s/(1 - \lambda) > 1$; $\lambda = 1$ is a pure integrator. Its policy is trained on relabelled demonstrations

$$u_t = \text{clip}((a_t - \lambda c_{t-1})/s, -1, 1), \quad c_t = \lambda c_{t-1} + s u_t, \quad (2)$$

computed *closed-loop* against the integrator’s own state so that clipping errors do not accumulate. Because u_t depends on c_{t-1} , which is not a function of the robot state, the label is not identifiable from the observation alone; in a pilot the diffusion prior without access to c_{t-1} fit the position dimensions with correlation ≈ 0 and achieved 0% success. We therefore append c_{t-1} to the observation for all derivative points (*observable state*). The low-pass points do not observe the filter state.

Proposition 1 (Observable-state interfaces are reparameterizations). *Substituting (2) into (1) gives $c_t = \lambda c_{t-1} + (a_t - \lambda c_{t-1}) = a_t$ whenever the clip is inactive. A policy that reproduces its labels therefore executes the demonstrated command exactly, for every (λ, s) : the interface is the position policy $\pi(a_t | o_t, c_{t-1})$ written in the output coordinates $(a_t - \lambda c_{t-1})/s$.*

Four corollaries organize the experiments. (i) *Intent is untouched*: the executed trajectory equals the intended one plus the filtered model error, so an observable-state interface can be smoother than the raw policy only by the amount the raw policy’s model error exceeds the demonstrations’, never smoother than the demonstrations. (ii) *The output is pre-whitened*: for a demonstration command with lag-1 autocorrelation ϕ , the label $(a_t - \lambda a_{t-1})/s$ has lag-1 autocorrelation $(\phi - \lambda)(1 - \lambda\phi)/(1 + \lambda^2 - 2\lambda\phi)$ under an AR(1) model—zero at $\lambda = \phi$, negative above, positive below—before any training. (iii) *Upstream corrections are disturbances*: a rescaling of u changes the map from output to c , which the policy observes and is trained to invert. (iv) *Impossibility*: a filter either is observed, and then cannot smooth intent, or is not, and then the derivative labels are not identifiable and the position prior is mismatched; smoothing of intent and a trainable derivative prior cannot coexist. The prior advantage of the derivative interface, whatever its size, must therefore come from conditioning on c_{t-1} and from the target rescaling by $1/s$, not from integration.

B. Three linear-system facts about perturbations

Let δu_t be a perturbation of the policy output (exploration noise, model error, or an adaptation correction) and δc_t its image on the executed command.

Proposition 2 (Spectral shaping of output perturbations). *For white δu_t with variance σ^2 and $\lambda < 1$, δc_t is AR(1) with stationary variance $s^2\sigma^2/(1 - \lambda^2)$ and low-frequency spectral exponent rising from 0 ($\lambda = 0$) to 2 ($\lambda \rightarrow 1$); for $\lambda = 1$ the variance grows as $s^2\sigma^2 t$.*

We call $\sigma_{\text{exec}} = s\sigma/\sqrt{1 - \lambda^2}$ the executed exploration noise. Because the policy replans every $H = 4$ steps, the perturbation it can react to is the displacement accumulated over a chunk; for an AR(1) process this is $\sigma_{\text{chunk}} = \sigma_{\text{exec}} \sqrt{\sum_{i,j < H} \lambda^{|i-j|}}$, which equals 2σ for white noise and $3.5\text{--}3.9\sigma_{\text{exec}}$ for $\lambda \in [0.8, 0.95]$.

Proposition 3 (Bounded propagation). *If $|\delta u_t| \leq \Delta$ for all t and $\lambda < 1$, then $|\delta c_t| \leq s\Delta/(1 - \lambda)$ and $|\delta x_t| \leq \|g\|_1 s\Delta/(1 - \lambda)$ for a tracking controller with impulse response g .*

Proposition 4 (Execution-space trust region). *$|c_t - c_{t-1}| \leq (1 - \lambda)C_{\text{max}} + s$ regardless of how far the policy parameters moved.*

Propositions 2–4 hold identically for the low-pass and the derivative interface; they depend on (λ, s) only, and—by Proposition 1—they constrain perturbations of the output, not the intended command. The “spectral exponent” of

Proposition 2 is a band-limited fit over the episode and chunk time-scales (the AR(1) spectrum is flat below its corner frequency $\approx (1 - \lambda)/2\pi$ and rolls off at exponent 2 above it); on that band the pink-noise optimum $\beta \approx 1$ [2] corresponds to $\lambda \approx 0.7$. With $C_{\text{max}} = 1$ the Proposition 4 bound is 0.4 for $(0.8, 0.2)$ and 1.1 for $(0.9, 1.0)$ —the original interface’s choice of $s = 1$ made its trust region barely tighter than the raw policy’s (2).

Two consequences frame the experiments. First, Proposition 2 applies to *any* perturbation injected upstream of the filter, but to *nothing* injected downstream; the placement of an adaptive correction therefore decides whether the filter protects it. Second, Propositions 2–4 say nothing about the policy’s own command spectrum, which by Proposition 1 is the demonstrations’; we return to this in Sec. VI-C.

IV. EVALUATION PROTOCOL

Each control below is stated with the false conclusion it prevented in this study.

- 1) **Matched-budget twin and strongest raw baseline.** Compare the interface both to the raw policy trained with the same policy-side σ (to isolate mechanism) and to the raw policy trained with its best σ (to measure deployment value). Against the twin, the interface showed +25–37 pp drift robustness and “made residual compensation usable”; against the strongest raw policy, both advantages reversed (Sec. VI-D, VI-E).
- 2) **Low-pass null hypothesis.** Include a unit-DC-gain filter at the same (λ, s) -bound. The low-pass arm turned out to be $4\times$ tighter on the execution-space trust region and $3\times$ smoother than the interface.
- 3) **Direct mechanistic measurement.** A proposed mechanism must come with a measurement that would refute it. “Integral action compensates gain loss” predicts an executed-magnitude ratio near 1 under gain drift; the measured ratio was 0.66 at gain 0.7 for both interface and raw policy—the interface did not wind up at all.
- 4) **Upstream versus downstream injection.** Perturb (and adapt) on both sides of the filter. Our first adaptation experiments applied $1/\hat{g}$ to the executed command, downstream of the integrator, and therefore measured the policy’s closed-loop robustness rather than the interface’s bound.
- 5) **Equivalence check for new estimator paths.** Before a new estimator enters a drift grid, verify on the no-drift condition that its activation rate and estimate trajectory match the old path step by step. A per-episode reset of the history buffer in our delay-aware estimator produced a plausible-looking “fixes the negative control, costs 20 pp” result that was entirely an artefact.

A sixth rule emerged from the map itself: **matched exploration budget must match** σ_{exec} , not σ . The seven-arm study that motivated this work matched σ and thereby compared an interface at $\sigma_{\text{exec}} = 0.069$ with a raw policy at 0.030.

V. EXPERIMENTAL SETUP

Task and pipeline. robomimic *Square* (nut assembly, mixed-human demonstrations) [21] in robosuite/MuJoCo [22], low-dimensional observations, OSC pose control at 20 Hz, action chunks of $H = 4$, 400-step episodes. Policies are DPPO diffusion MLPs [3]: a diffusion prior pre-trained for 3000 epochs (stock prior for raw and low-pass points; a prior trained on relabelled data (2) for derivative points), then 200 PPO iterations with the final-step sampling noise σ as the exploration parameter. Training seed 42 for all points; seeds 43–46 additionally for E4 and E5.

Points. Table I lists the twelve configurations. A, E5, B, E1, E4 come from the original seven-arm study; F1–F7 were trained for the map. F4/F7 and F5 share the (0.8, 0.2) filter with B and form a $\{\text{prior}\} \times \{\sigma\}$ factorial; F6 is F1’s prior fine-tuned at $\sigma = 0.07$.

Axes and metrics. (1) *Prior alignment*: success of the pre-trained prior through the interface before RL (200 episodes). (2) *Trainability*: success after 200 iterations. (3) *Smoothness*: mean executed jerk and maximum per-step change of the executed position command, deterministic rollouts. (4) *Deployment robustness*: frozen success under actuator gain $\times 0.7$, a two-step control delay, and observation noise $\sigma_{\text{obs}} = 0.05$. (5) *Signal and adaptation*: correlation of the free-motion tracking residual with the command (identifiability of the nominal map K), recovery of a residual-driven gain compensation under gain $\times 0.7$, and the *misadaptation cost*—success change when the same estimator is fooled by a delay it misreads as a gain loss. Axes 3–5 use three evaluation seeds $\times 100$ episodes per cell.

VI. RESULTS

A. Prior alignment: smaller s is better

The derivative prior is better than the stock prior for every (λ, s) we trained (49.0–59.5% versus 38.0%), and—contrary to our pre-registered expectation that a small s would clip demonstration increments and hurt alignment—it improves as s shrinks: at $\lambda = 0.9$, $s = 1.0 \rightarrow 0.3$ raises the prior from 49.0 to 58.5%; at $s = 1$, $\lambda = 0.9 \rightarrow 0.7$ raises it from 49.0 to 54.0%. The mechanism is label range: with $s = 1$ the closed-loop labels (2) have mean magnitude $|u| \approx 0.09$ and are dominated by the diffusion model’s output noise; with $s = 0.3$ they fill the $[-1, 1]$ range while clipping fewer than 1% of steps. The low-pass arms, which use the stock prior through an unmatched filter, start at 23.5%. Retraining the prior for the same (0.8, 0.2) filter with an observable state (F5) gives 59.5%, identical to the best derivative point: on this axis the effect is entirely due to retraining and observability, not to DC gain.

B. Trainability: an inverted U in σ_{exec} and seed variance

Final success is high (94–99.5%) for every point with $\sigma_{\text{exec}} \in [0.02, 0.10]$ and collapses outside it: the low-pass arm at $\sigma = 0.03$ (F4, $\sigma_{\text{exec}} = 0.010$) reaches only 77.5%, the derivative arms at $\sigma = 0.10$ (E3 at $\sigma_{\text{exec}} = 0.23$, C with a pure integrator) collapsed to 12.5 and 24.5% in the original

study (Fig. 2, left). The filter’s noise gain, not its structure, sets the usable σ : a low-pass attenuates policy-side noise by $(1 - \alpha)/(1 + \alpha) = 0.11$ in variance and under-explores at $\sigma = 0.03$; a leaky integrator with $s = 1$ amplifies it by $1/(1 - \lambda^2) = 5.3$ and over-explores at $\sigma = 0.10$. The original interface at $\sigma = 0.03$ sits at the right edge of the sweet spot; across five training seeds its final success is $92.5 \pm 8.5\%$ (one seed at 77.5%) versus $92.1 \pm 1.3\%$ for the raw twin—equal mean, $6.5\times$ the standard deviation. The prior also matters for trainability: at equal filter and σ , the retrained-prior point F5 reaches 94.0% where the stock-prior F4 reaches 77.5%.

C. Smoothness: the bound holds, but the observable state whitens the policy

The maximum per-step change of the executed command tracks the Proposition 3 bound across all twelve points (Table I: 0.14–0.33 for bound 0.4, 0.43 for 0.55, 0.60 for 1.3, 0.67–0.79 for bounds 1.1–2.0). Within a bound, however, the points split by observability. Table II reports the lag-1 autocorrelation of the policy output and of the executed command. Policies that cannot see the filter state (B, F4) produce output as smooth as the raw policy (0.78–0.81) and the filter adds genuine smoothing on top: executed jerk 0.026–0.032, spectral exponent $\beta_c \approx 3$. Policies that see the filter state produce output that is white or anti-correlated (-0.14 to 0.40 ; sign-flip rate 0.67–0.73 versus 0.60), and their executed jerk is $2.8\text{--}3.2\times$ that of the unobservable twin at the same filter and σ (F4→F5: 0.026→0.074; B→F7: 0.032→0.101); their executed spectral exponent is 1.8–2.25, and E4’s (1.83) is indistinguishable from the raw policy’s (1.76).

This is Proposition 1 at work, not something RL learned. The demonstrations themselves have lag-1 autocorrelation $\phi = 0.91$, spectral exponent 1.93 and jerk 0.107 (300 trajectories, position dimensions). The relabelled datasets already carry the pre-whitened labels of corollary (ii): their lag-1 autocorrelation is -0.08 at $\lambda = 0.9$, -0.14 at $\lambda = 0.95$, $+0.35$ at $\lambda = 0.7$ (AR(1) prediction $+0.35$) and $+0.24$ at $\lambda = 0.8$, and the fine-tuned policies reproduce these values (-0.14 , -0.11 , $+0.40$, $+0.25\text{--}0.31$ in Table II). The integrator then re-reddens the output and the executed command recovers the demonstrations’ statistics: executed jerk 0.07–0.13 against the demonstrations’ 0.107 and exponent 1.8–2.25 against 1.93. The raw policy is *jerkier* than the demonstrations (0.16–0.18) because its model error is executed unfiltered; the observable-state interface removes that excess and no more; only the unobservable low-pass smooths the intent itself (0.03). The claim “derivative interfaces execute more smoothly” is therefore true of *perturbations*—exploration noise (Proposition 2; in stochastic rollouts the executed exponent of E4 rises to 2.1–2.5 against 0.9 for the raw twin) and model error—and false of the *intended trajectory*.

D. Deployment robustness: a function of executed noise

Under a 30% actuator-gain loss, frozen success ranges from 1% to 81% across the map, and the ordering is explained by the training noise, not by the interface (Fig. 2, right). Against

TABLE I

THE MAP, $\sigma_{\text{exec}} = s\sigma/\sqrt{1-\lambda^2}$ ($\lambda = 1$: END-OF-EPISEODE RANDOM-WALK STD). PRIOR AND FINAL SUCCESS FROM TRAINING SEED 42; DEPLOYMENT COLUMNS ARE MEANS OVER THREE EVALUATION SEEDS $\times 100$ EPISODES. OBS.: FILTER STATE OBSERVED BY THE POLICY. P3: BOUND $(1-\lambda) + s$.

	λ	s	σ	Obs.	DC	σ_{exec}	Prior	Final	Jerk	Max step (P3)	Gain $\times 0.7$	Delay 2	Obs. noise	Calib. ρ	Recover	F
A	0	1.0	.10	–	1	.100	38.0	99.5	.17	.68 (2.0)	81.3	91.0	–	.81	+17.0	–
E5	0	1.0	.03	–	1	.030	38.0	93.0	.16	.67 (2.0)	3.7	80.0	49	.75	+40.7	–
B	.8	.2	.10	no	1	.033	23.5	96.0	.03	.20 (0.4)	41.0	87.0	–	.93	+55.7	–
F4	.8	.2	.03	no	1	.010	23.5	77.5	.03	.14 (0.4)	1.0	64.7	–	.80	+39.7	–
F5	.8	.2	.03	yes	1	.010	59.5	94.0	.07	.26 (0.4)	4.0	73.3	–	.94	+79.0	–
F7	.8	.2	.10	yes	1	.033	59.5	99.5	.10	.29 (0.4)	28.7	86.7	–	.73	+19.7	–
F1	.9	.3	.03	yes	3	.021	58.5	94.0	.07	.30 (0.4)	12.0	74.7	–	.89	+73.7	–
F6	.9	.3	.07	yes	3	.048	58.5	99.5	.09	.33 (0.4)	38.3	67.3	89	.90	+56.3	–
F2	.7	1.0	.03	yes	3.3	.042	54.0	98.0	.11	.60 (1.3)	38.7	80.0	–	.93	+58.7	–
F3	.95	.5	.03	yes	10	.048	59.5	99.0	.10	.43 (0.55)	45.7	84.0	84	.85	+48.7	–
E4	.9	1.0	.03	yes	10	.069	49.0	95.0	.13	.79 (1.1)	28.3	87.3	78	.75	+62.7	–
E1	1.0	1.0	.03	yes	∞	.60	43.5	92.0	.12	.76 (1.0)	45.3	83.0	76	–	–	–

TABLE II

COMMAND STATISTICS ON DETERMINISTIC NO-DRIFT ROLLOUTS (POSITION DIMENSIONS, THREE EVALUATION SEEDS). AC₁: LAG-1 AUTOCORRELATION; β : LOW-FREQUENCY SPECTRAL EXPONENT.

Point	State	ac ₁ (u)	ac ₁ (c)	flips	jerk	β_c
A	raw	.80	.80	.59	.175	1.48
E5	raw	.77	.77	.60	.161	1.76
B	unobserved	.81	.99	.61	.032	3.04
F4	unobserved	.78	.99	.61	.026	2.85
F5	observed	.31	.96	.70	.074	1.98
F7	observed	.25	.96	.70	.101	2.25
E4	observed	-.14	.93	.71	.134	1.83
F1	observed	-.11	.95	.73	.073	2.08
F6	observed	.03	.96	.72	.086	2.14
F2	observed	.40	.94	.67	.115	2.01
F3	observed	-.11	.94	.71	.098	1.78

σ_{exec} , Spearman’s $\rho = 0.76$ over the ten stationary points; against the chunk-scale, colour-aware σ_{chunk} it is also 0.76, rising to 0.92 over the nine points that exclude the original interface E4 (we report both because E4 is the configuration that motivated the study). The chunk-scale variant explains why the raw policy at $\sigma = 0.03$ is as fragile as the $\sigma = 0.03$ low-pass arms (3.7% versus 1–4%); white noise of standard deviation 0.03 accumulates to only 0.06 within one decision period, too little for the policy ever to observe and correct a deviation, whereas the same σ_{exec} passed through a $\lambda = 0.8$ filter accumulates to 0.12 (B: 41%).

Two configurations sit below the trend, both with observable state: E4 ($\sigma_{\text{chunk}} = 0.26$, 28.3%) and F7 ($\sigma_{\text{chunk}} = 0.12$, 28.7% against 41% for its unobservable twin B). Neither is training-seed noise: across five training seeds E4’s gain-drift success is $25.9 \pm 7.5\%$ and E5’s $4.3 \pm 2.7\%$. We tested and rejected three explanations. Prior quality: F5 and F6, with the best priors on the map, land inside the range predicted from σ_{exec} alone (4.0% and 38.3%). Training-seed variance: above. Noise scale and colour: the residual survives both metrics. What remains is that a policy observing c_{t-1} relies on the consistency between the command it observes and the motion

it observes, which a gain change breaks; the other observable-state points (F1, F2, F3, F5, F6) sit on the trend, so this cannot be the whole story, and we leave it open. What is not open is the ranking: the raw policy trained with $\sigma = 0.10$ (81.3%) is more robust than any interface configuration (at most 45.7%), and the interface’s apparent +25 pp advantage over its matched- σ twin is the difference between two points that both lie below the curve.

Robustness to a two-step control delay behaves differently: it runs *opposite* to s among the $\sigma = 0.03$ derivative points (73–75% at $s \leq 0.3$, 84% at $s = 0.5$, 80–87% at $s = 1$) and drops further when σ is raised (F6: 67%), while the raw policy at $\sigma = 0.10$ reaches 91%. A smaller s makes the policy output larger and more feedback-dependent, and a delay corrupts exactly that feedback; we report it as a separate sub-axis.

E. Signals and adaptation: where the correction is applied decides everything

We estimate the actuator gain ratio $\hat{g}$ from free-motion tracking residuals with a sliding-window least squares against a per-policy calibrated nominal map K , and apply $1/\hat{g}$ with a dead-band trigger. Two facts organise the results (Fig. 3).

Downstream compensation measures the policy, not the interface. Applied to the executed command, the same estimator recovers every arm: the raw $\sigma = 0.10$ policy from 81 to 99% with a misadaptation cost of -0.3 pp under no drift, the interface from 28 to 91%, the low-pass from 41 to 97%. The matched- σ raw twin recovers only to 44% and loses 38–48 pp when the estimator is active without drift—but it is a fragile policy, not a policy without an interface. When the estimator is fooled by a two-step delay it misreads as $\hat{g} \approx 0.65$, the strong raw policy loses 37–42 pp and doubles its jerk while the interface loses 11–25 pp; at an *equal* injected gain error, however ($\times 1.4 / \times 1.6$ on frozen policies), the raw policy keeps 88/67% and the interface 68/33%—the crossover was a difference in how badly each arm’s estimator was fooled, not a structural protection.

Upstream compensation is protected by the filter but is absorbed by an observable-state policy. Applied to the

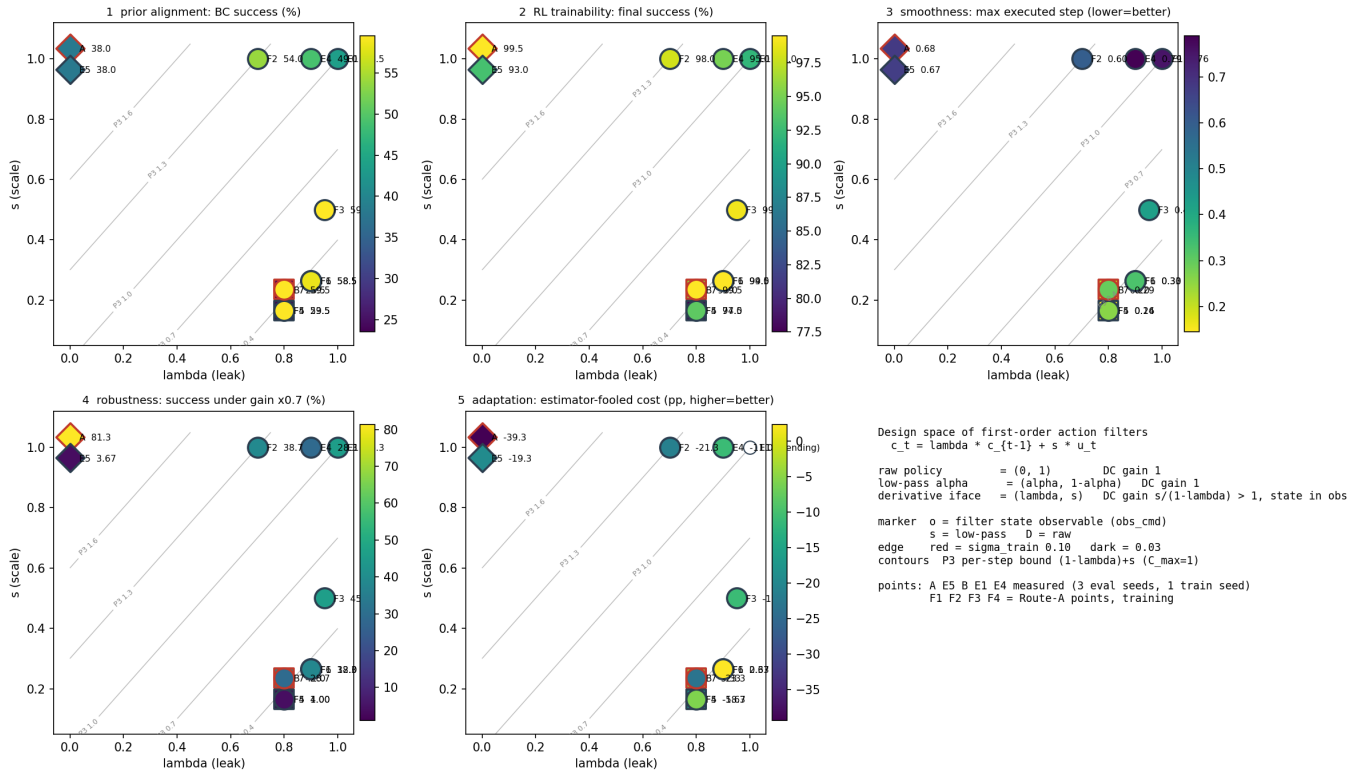


Fig. 1. The (λ, s) design space, one panel per axis. Raw policies are the point $(0, 1)$; low-pass filters lie on $s = 1 - \lambda$; derivative interfaces have $s/(1 - \lambda) > 1$. Marker: circle = filter state observed, square = low-pass, diamond = raw; edge colour: training $\sigma = 0.10$ (red) or 0.03 (dark). Grey contours: the Proposition 3 bound $(1 - \lambda) + s$. Smoothness (panel 3) follows the bound; robustness (panel 4) follows the edge colour, i.e. the training noise.

policy output, so that Proposition 2 applies, the correction recovers the low-pass arm exactly as well as downstream (92–93% versus 94–97%) while injected white adaptation noise of relative standard deviation 0.5 raises executed jerk by only $1.40\times$ upstream against $6.11\times$ downstream—the structural bound is real. On the derivative interface the same upstream correction does *not* recover gain drift (27–36% versus 87–91% downstream) and makes execution *jerkier* (1.7 – $2.3\times$): the policy observes c_{t-1} , regulates c rather than u , and undoes the upstream scaling at the next decision, fighting it (corollary (iii) of Proposition 1). For an observable-state interface the parameters can therefore only be adapted downstream, where the filter offers no protection. Zero-mean adaptation noise, finally, barely affects success for any arm (≤ 6 pp at relative std 0.5); what costs success is bias, and bias is placement-invariant.

Signal quality follows command colour: the calibration correlation is highest for the unobservable low-pass (0.93) and the observable-state point with the smoothest output (F5, 0.94) and lowest for the whitened E4/E5/F7 (0.73–0.75); the smoothed commands of all filtered arms make a delay harder to distinguish from a gain loss (Sec. VI-D).

F. Putting the axes together

No point dominates. The strongest raw policy (A) wins zero-shot robustness (81%), delay robustness (91%) and

final success (99.5%) but has the worst prior (38%), the largest misadaptation cost (-39 pp) and raw-policy jerk. The unobservable low-pass (B) wins smoothness and the execution-space bound by $4\times$ and has the smallest misadaptation cost among the $\sigma = 0.10$ arms, but starts from the worst prior (23.5%). The best derivative configuration we found, F6 $= (0.9, 0.3)$ at $\sigma = 0.07$, improves on the original interface E4 on every axis but delay—prior $+9.5$ pp, final $+4.5$, gain robustness $+10$, observation-noise robustness $+11$, maximum step 0.33 versus 0.79, misadaptation cost $+2$ versus -11 —by moving two of E4’s three knobs: s was set to 1 on a mistaken belief about prior alignment, and σ to 0.03 because the interface’s noise gain left no choice. F6 still loses to A on robustness (38 versus 81%).

What the derivative interface buys, on this task, is therefore: a better behaviour-cloning prior (via retraining with an observable state and a small s) and coloured exploration noise. What it does not buy: smoother intended trajectories (the observable state whitens the policy), better fine-tuning (equal mean, $6.5\times$ the variance), zero-shot robustness (governed by σ_{exec} , which the interface caps), usable adaptation (only downstream, unprotected), or better self-supervised signals.

VII. DISCUSSION AND LIMITATIONS

Why the first study was wrong. Every advantage in our seven-arm pilot came from comparing against a raw policy

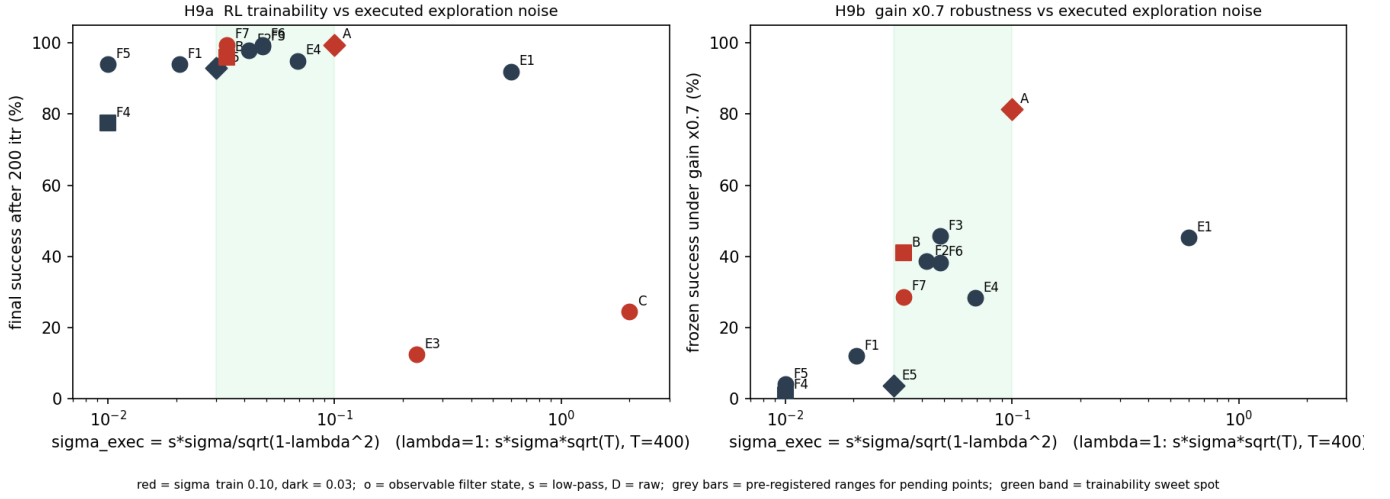


Fig. 2. Trainability (left) and gain-drift robustness (right) against the executed exploration noise σ_{exec} . Left: an inverted U with a sweet spot around 0.03–0.10. Right: robustness rises monotonically with σ_{exec} except for two configurations with observable state (E4, F7) that sit below the trend; both are stable across training seeds.

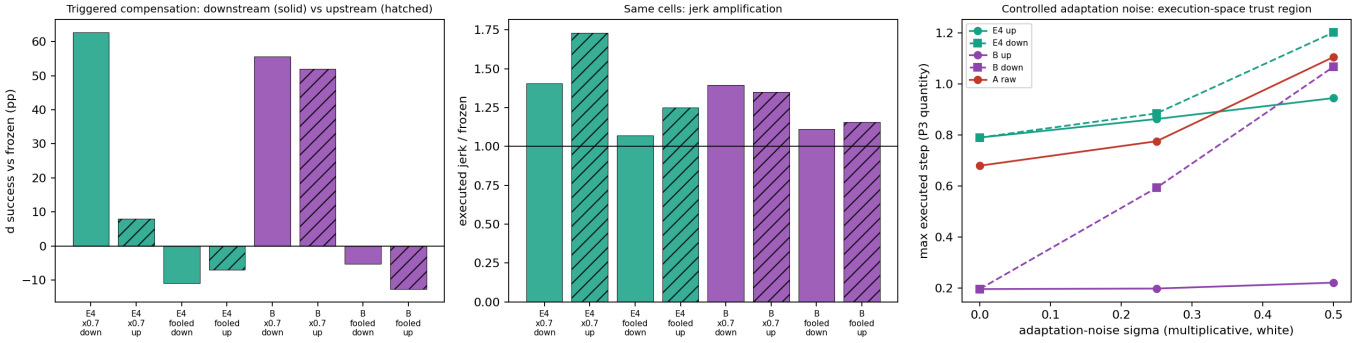


Fig. 3. Placement of the adaptive correction. Left/centre: triggered gain compensation applied downstream (solid) versus upstream (hatched) of the filter under gain $\times 0.7$ and under an estimator fooled by delay; on the low-pass arm (B) both placements work, on the observable-state interface (E4) only downstream does. Right: maximum executed step under injected white adaptation noise; the low-pass arm with upstream injection stays within its Proposition 3 bound (0.22 at relative std 0.5) while downstream injection and the interface do not.

fine-tuned at $\sigma = 0.03$. That twin is the correct control for *mechanism*—it isolates the interface—but the wrong control for *value*, because the interface’s own noise gain is what forced $\sigma = 0.03$ in the first place. The protocol of Sec. IV exists so that the two questions are answered by different comparisons.

On “smoother” action spaces. Propositions 2–4 are correct and are the usual justification for derivative parameterizations. They constrain perturbations, not the policy’s command; by Proposition 1 the relabelling that makes a derivative policy trainable also fixes its executed command to the demonstrations’. Any claim of smoother execution should be checked on deterministic rollouts against an unobservable low-pass filter of the same bound.

Limitations. One task (quasi-static insertion), one simulator, one RL algorithm, low-dimensional observations. Most points have a single training seed; the two principal arms have five. The open residual of Sec. VI-D (E4 and F7 below the σ_{exec} trend) is unexplained. Tasks where

smoothness is a constraint rather than a preference (liquids, fragile contact) may reward the filtered arms differently; there the unobservable low-pass, not the derivative interface, is the natural candidate.

VIII. CONCLUSION

We recast derivative, velocity and low-pass action interfaces as points of one first-order filter family and measured a diffusion-policy RL pipeline across that family on five axes. The interface that motivated the study buys a better prior and coloured exploration noise; its remaining claimed benefits are either properties of the executed exploration noise—which any filter sets and which the derivative interface sets badly—or are forfeited by the observable filter state that its labels require. We hope the map and the protocol make the next action-space proposal cheaper to evaluate and harder to over-claim.

REPRODUCIBILITY

Code, configurations, the drift and misadaptation suites, and all rollout statistics are released with the paper.[\[URL\]](#)

REFERENCES

- [1] A. Raffin, J. Kober, and F. Stulp, “Smooth exploration for robotic reinforcement learning,” 2021. [Online]. Available: <https://arxiv.org/abs/2005.05719>
- [2] O. Eberhard, J. Hollenstein, C. Pinneri, and G. Martius, “Pink noise is all you need: Colored noise exploration in deep reinforcement learning,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://openreview.net/forum?id=hQ9V5QN27eS>
- [3] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz, “Diffusion policy policy optimization,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.00588>
- [4] Q. Li, Z. Zhou, and S. Levine, “Reinforcement learning with action chunking,” 2026. [Online]. Available: <https://arxiv.org/abs/2507.07969>
- [5] M. A. Muttaqien, T. Motoda, R. Hanai, and Y. Domae, “Orpa: Online residual policy adaptation for robot manipulation control with human feedback,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.17323>
- [6] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, “Steering your diffusion policy with latent space reinforcement learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.15799>
- [7] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023.
- [8] W. Chen, K. Zhang, C. Lin, Z. Zhang, Y. She, Y. Liu, R. A. Yeh, S. Mou, and Y. Gu, “Dream-chunk: Reactive action chunking with latent world model,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.18589>
- [9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” in *arXiv preprint arXiv:1707.06347*, 2017.
- [10] M. Nakamoto, Y. Zhai, A. Singh, M. S. Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine, “Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.05479>
- [11] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, “Efficient online reinforcement learning with offline data,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.02948>
- [12] Z. Zhou, A. Peng, Q. Li, S. Levine, and A. Kumar, “Efficient online reinforcement learning fine-tuning need not retain offline data,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.07762>
- [13] M. O’Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, “Neural-fly enables rapid learning for agile flight in strong winds,” 2024. [Online]. Available: <https://arxiv.org/abs/2205.06908>
- [14] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “Rma: Rapid motor adaptation for legged robots,” 2021. [Online]. Available: <https://arxiv.org/abs/2107.04034>
- [15] N. Hansen, R. Jangir, Y. Sun, G. Alenyà, P. Abbeel, A. A. Efros, L. Pinto, and X. Wang, “Self-supervised policy adaptation during deployment,” 2021. [Online]. Available: <https://arxiv.org/abs/2007.04309>
- [16] Y. Jiang, Y. Chebotar, R. Zheng, F. Hu, Y. Ge, J. Wu, T. Dai, S. Reed, L. Fei-Fei, Y. Zhu, and L. J. Fan, “Robottt: Context scaling for robot policies,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.15275>
- [17] C. Sima, K. Chitta, Z. Yu, S. Lan, P. Luo, A. Geiger, H. Li, and J. M. Alvarez, “Centaur: Robust end-to-end autonomous driving with test-time training,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.11650>
- [18] W. Zhang, J. Li, S. Yang, S. Chen, J. Liu, L. Liu, and X. Ma, “Ttt-vla: Test-time latent prompt optimization for vision-language-action models,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.03127>
- [19] O. Press, S. Schneider, M. Kümmerer, and M. Bethge, “Rdumb: A simple approach that questions our progress in continual test-time adaptation,” 2024. [Online]. Available: <https://arxiv.org/abs/2306.05401>
- [20] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, “Parting with misconceptions about learning-based vehicle motion planning,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.07962>
- [21] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Conference on Robot Learning (CoRL)*, 2021.
- [22] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, S. Nasiriany, and Y. Zhu, “robosuite: A modular simulation framework and benchmark for robot learning,” in *arXiv preprint arXiv:2009.12293*, 2020.